Unit-5: Graph Theory

Basic Terms (Foundat on)

Graph theory models **relat onships between objects**. It is widely used in computer science (networks, social graphs, rout ng, etc.).

---

1 Graph

Def nit on : A graph is a mathemat cal structure consist ng of a set of vert ces (nodes) and a set of edges that connect pairs of vert ces.

Notat on

$$G=(V,E)$$

Where:

Symbol	Meaning
V	Set of vert ces (nodes)
E	Set of edges

Example

Let:

V = {A, B, C}
E = {(A,B), (B,C)}

Graph representat on:

A —— B —— C

---

2 Nodes / Vert ces

Def nit on : A vertex (or node) is a fundamental unit of a graph that represents an object or ent ty.

Example

In a social network:

- Each person = vertex
- Friendship = edge

Example graph:

V = {1,2,3,4}

Each number represents a **vertex**.

3 Edge

Def nit on : An edge is a connect on between two vert ces in a graph.

Example:

If vertex A is connected to B:

Edge = (A,B)

Graph: A —— B

Edge represents the relat onship.

Types of Edges

Type	Meaning
Undirected Edge	Connect on without direct on
Directed Edge	Connect on with direct on

Examples:

Undirected : A —— B

Directed : A → B

---

4 Adjacent Nodes (Adjacent Vert ces)

Def nit on : Two vert ces are said to be adjacent if they are connected by an edge.

Example

Graph:

A —— B —— C

Adjacency:

Vertex	Adjacent To
A	B
B	A, C
C	B

So:

- A and B are adjacent
- B and C are adjacent
- A and C are **not adjacent**

---

5 Endpoints of an Edge

Def nit on : The vert ces connected by an edge are called endpoints of that edge.

Example:

Edge (A,B)

Endpoints = A and B.

---

6 Order and Size of a Graph

Term	Meaning
Order	Number of vert ces
Size	Number of edges

Example:

V = {A,B,C,D}
E = {(A,B),(B,C),(C,D)}

Order = 4
Size = 3

---

1 Undirected Graph

Def nit on : An undirected graph is a graph in which edges have no direct on; the connect on between two vert ces is mutual.

Notat on

If vert ces **A** and **B** are connected:

(A,B) = (B,A)

Meaning: Connect on works both ways.

Example

Vert ces:

V = {A, B, C}

Edges:

E = {(A,B), (B,C)}

Graph:

A —— B —— C

Meaning:

- A connected to B
- B connected to C

Connect ons are **two-way**.

Real Life Example

- Friendship network

- Road between two cit es
- Communicat on between computers

If A connects to B → B connects to A.

---

2 Directed Graph (Digraph)

Def nit on : A directed graph is a graph in which each edge has a direct on, represented by an ordered pair of vert ces.

Notat on

$$(A, B) \neq (B, A)$$

Meaning:
Edge goes from A to B only.

Example

Vert ces:

V = {A, B, C}

Edges:

E = {(A,B), (B,C)}

Graph:

A → B → C

Meaning:

- A points to B
- B points to C

Connect on has **direct on**.

Real Life Example

- Twit er follow system
- Web page links
- One-way streets

If A follows B → B may not follow A.

Key Dif erences

Feature	Undirected Graph	Directed Graph
Edge Direct on	No direct on	Has direct on

Edge Representat on	(A,B) = (B,A)	(A,B) ≠ (B,A)
Arrow	No arrow	Arrow present
Example	Facebook friends	Twit er follow

Small Example Comparison

Undirected

Edges:

{(A,B),(B,C)}

Graph:

A —— B —— C

Directed

Edges:

{(A,B),(B,C)}

Graph:

A → B → C

---

3 In-Degree of a Node

Def nit on : The in-degree of a vertex in a directed graph is the number of edges directed toward that vertex.

Notat on

$$deg^{-}(v)$$

Example

If edges are:

A → B
C → B

Then:

$$deg^{-}(B) = 2$$

Two edges enter node **B**.

Out-Degree of a Node

Def nit on : The out-degree of a vertex in a directed graph is the number of edges leaving that vertex.

Notat on

$$deg^{+}(v)$$

Example

If edges are:

A → B
A → C

Then:

$$deg^{+}(A) = 2$$

Two edges start from **A**.

---

5 Total Degree of a Node

Def nit on : The total degree of a vertex in a directed graph is the sum of its in-degree and out-degree.

Formula

$$deg(v) = deg^{-}(v) + deg^{+}(v)$$

Example

Edges:

A → B
B → C
C → B

For vertex B:

In-degree = 2
Out-degree = 1

Total degree = 3

---

6 Null Graph

Def nit on: A null graph is a graph that has vert ces but no edges.

Example

V = {A, B, C}

E = Ø

Graph:

A B C

(no connect ons)

---

7 Trivial Graph

Def nit on : A trivial graph is a graph that contains exactly one vertex and no edges.

Example

V = {A}
E = Ø

Graph:

A

---

8 Loop (Self-Edge)

Def nit on : A loop is an edge that connects a vertex to itself.

Example

Edge: (A, A)

Graph:

A ↺

Meaning vertex connects to itself.

---

9 Isolated Vertex

Def nit on : A vertex that has no edges connected to it is called an isolated vertex.

Property

Degree = 0

Example

Graph:

A —— B C

Here **C** is isolated.

---

1 Dist nct Edges

Def nit on : Dist nct edges are edges that connect dif erent pairs of vert ces.

Example:

(A,B)
(B,C)

These edges connect dif erent vert ces.

---

2 Parallel Edges

Def nit on : Parallel edges are two or more edges that connect the same pair of vert ces.

Example

A ==== B

Two edges between A and B.

Such graphs are called **mult graphs**.

---

3 Init at ng and Terminat ng Nodes

(Used in directed graphs)

Init at ng Node

The vertex from which a directed edge starts.

Terminat ng Node

The vertex where the directed edge ends.

Example

Edge:

A → B

Node	Role
A	Init at ng node
B	Terminat ng node

---

4 Simple Graph

Def nit on : A simple graph is an undirected graph that contains no loops and no parallel edges.

Propert es

- Only one edge between any two vert ces
- No vertex connected to itself

Example

Vert ces
V = {A, B, C}

Edges
E = {(A,B), (B,C)}

Graph:

A —— B —— C

---

5 Mult graph

Def nit on : A mult graph is a graph in which two or more edges can connect the same pair of vert ces.

These edges are called **parallel edges**.

Example

A ==== B

Two edges between A and B.

Used when mult ple relat onships exist between same nodes.

Example: mult ple roads between cit es.

---

6 Pseudograph

Def nit on: A pseudograph is a graph that may contain both loops and parallel edges.

Propert es

- Loops allowed
- Parallel edges allowed

Example

A ↺
A ==== B

Loop on A and mult ple edges between A and B.

---

7 Weighted Graph

Def nit on: A weighted graph is a graph in which each edge is assigned a numerical value called weight.

Weights represent:

- distance
- cost
- t me
- capacity

Example

A --5-- B --3-- C

Edge weights:

- A–B = 5
- B–C = 3

Used in shortest path algorithms.

---

8 Subgraph

Def nit on : A subgraph is a graph whose vert ces and edges are subsets of another graph.

If:

$$G = (V,E)$$

Then subgraph:

$$H = (V', E')$$

Where

$$V' \subseteq V$$

$$E' \subseteq E$$

Example

Original graph:

A —— B —— C
|
D

Subgraph:

A —— B

---

9 Isomorphic Graphs (☆ important topic)

Def nit on : Two graphs are isomorphic if they have the same structure but their vert ces are labeled dif erently.

Condit ons

Two graphs are isomorphic if they have:

1. Same number of vert ces
2. Same number of edges
3. Same vertex degrees
4. Same connect vity pat ern

Example

Graph 1

A —— B
| |
D —— C

Graph 2

1 —— 2
| |
4 —— 3

Although labels dif er, structure is same → **Isomorphic graphs**.

---

1 Def nit on of Isomorphic Graph

Exam Def nit on

Two graphs are said to be **isomorphic** if there exists a one-to-one correspondence between their vertex sets such that adjacency between vert ces is preserved.

In simple words:

Two graphs are **structurally ident cal**, only the **vertex names are dif erent**.

---

2 Mathemat cal Representat on

If:

$$G_1 = (V_1, E_1)$$

$$G_2 = (V_2, E_2)$$

Graphs are isomorphic if there exists a **biject ve funct on**

$$f: V_1 \to V_2$$

such that

$$(u, v) \in E_1 \iff \big(f(u), f(v)\big) \in E_2$$

Meaning: connect ons remain the same af er relabeling.

---

3 Example of Isomorphic Graphs

Graph G_1

Vert ces:

A, B, C, D

Edges:

(A,B), (B,C), (C,D), (D,A)

Graph:

A —— B
| |
D —— C

Graph G_2

Vert ces:

1, 2, 3, 4

Edges:

(1,2), (2,3), (3,4), (4,1)

Graph:

1 —— 2
| |
4 —— 3

Mapping:

G_1	G_2
A	1
B	2
C	3
D	4

Edges remain same → **Graphs are isomorphic**

---

4 Condit ons for Isomorphic Graphs

Two graphs must sat sfy these condit ons.

Condit on	**Explanat on**
Same number of vert ces	
Same number of edges	
Same degree sequence	Vertex degrees ident cal
Same adjacency relat onships	Connect vity pat ern same

---

5 Degree Sequence Method (Important for Exams)

Step 1
Write degree of each vertex.

Step 2
Sort them.

Step 3
Compare both graphs.

If sequences dif er → **Not isomorphic**

Example

Graph G_1 degrees:

3, 2, 2, 1

Graph G_2 degrees:

3, 2, 2, 1

Since equal → **possibly isomorphic**

Then check adjacency.

---

6 When Graphs Are NOT Isomorphic

Graphs cannot be isomorphic if:

- Number of vert ces dif ers
- Number of edges dif ers
- Degree sequences dif er
- Connect vity pat ern dif ers

Example

Graph 1

A —— B —— C

Degrees:

1,2,1

Graph 2

1 —— 2

Degrees:

1,1

Not same → **Not isomorphic**

---

7 Steps to Check Graph Isomorphism (Exam Method)

1 Count vert ces
2 Count edges
3 Compare degree sequence
4 Compare adjacency structure
5 Try vertex mapping

If all match → isomorphic.

---

8 Real Life Meaning

Isomorphic graphs represent **same network with dif erent labels**.

Example:

- Computer networks
- Chemical structures
- Social network structures

---

9 Important Observat on

Isomorphic graphs:

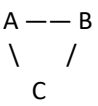
- Look dif erent visually

- But are structurally ident cal.

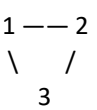
---

10 Quick Example

Graph 1



Graph 2



Mapping:

A → 1
B → 2
C → 3

Graphs are **isomorphic**.

What Professors Usually Ask

1 Def ne isomorphic graph
2 Check if two graphs are isomorphic
3 Write vertex mapping
4 Compare degree sequences

Graph Theory – Paths and Related Terms (Unit 5)

These concepts describe **how vert ces are connected and traversed** in a graph.

1 Walk

Def nit on : A walk in a graph is a sequence of vert ces and edges where each edge connects the preceding vertex to the following vertex.

Vert ces and edges **may repeat**.

Example

Graph: A — B — C — D

Walk: A → B → C → B → D

Here vertex **B repeats**, so it is st ll a walk.

2 Path

Def nit on A path is a walk in which all edges are dist nct (no edge is repeated). Vert ces may repeat, but edges cannot repeat.

Example

Graph:

A — B — C — D

Path:

A → B → C → D

---

3 Length of a Path

Def nit on : The length of a path is the number of edges in the path.

Example

Path: A → B → C → D

Edges:

(A,B), (B,C), (C,D)

Length = **3**

---

4 Paths of a Given Graph

These are **all possible paths between vert ces** in a graph.

Example graph:

A — B — C

Possible paths:

- A → B
- B → C
- A → B → C

---

5 Simple Path (Edge-Simple Path)

Def nit on : A simple path is a path in which no edge is repeated.

Vert ces may repeat but edges cannot.

Example:

A → B → C → B → D
(Edges are dif erent)

---

6 Elementary Path (Node-Simple Path)

Def nit on : An elementary path is a path in which no vertex is repeated.

Example

A → B → C → D

Each vertex appears only once.

Dif erence

Type	Edge Repet t on	Vertex Repet t on
Walk	Allowed	Allowed
Path	Not allowed	Allowed
Elementary Path	Not allowed	Not allowed

---

7 Circuit

Def nit on: A circuit is a path that starts and ends at the same vertex.

Edges cannot repeat. But vertex are repeated

Example

A → B → C → A

Start = End = A

---

8 Cycle

Def nit on : A cycle is a closed path in which the start ng and ending vert ces are the same and no other vert ces are repeated.

Example

A → B → C → A

Vert ces B and C appear once.

---

9 Elementary Cycle

Def nit on: An elementary cycle is a cycle in which no vertex is repeated except the start ng and ending vertex.

Example:

A → B → C → D → A

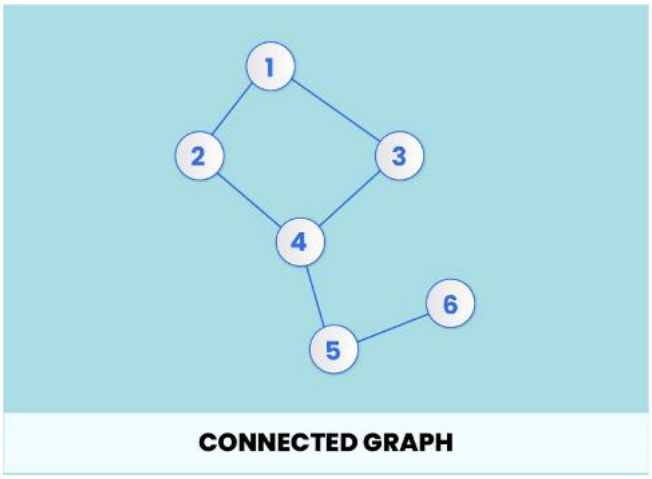
Notes

- **Circuit:** Closed path with no repeated edges, but vert ces may repeat.
- **Cycle:** Closed path where **no vertex repeats** except the f rst/last vertex.
- **Elementary Cycle:** Same condit on as cycle; no vertex repet t on except the start/end vertex.

---

10 **Converse (Reversal) of a Digraph**

Def nit on : The converse of a directed graph is obtained by reversing the direct on of every edge in the graph.



Example

Original digraph:

A → B
B → C

Converse graph:

A ← B
B ← C

or

B → A
C → B

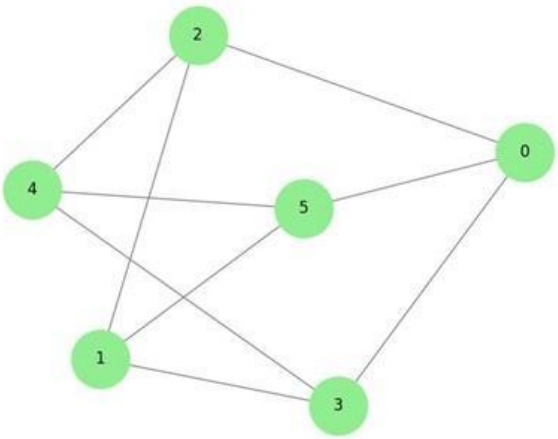
---

1. Regular Graph

Def nit on : A **regular graph** is a graph in which every vertex has the same degree.

If every vertex has degree **k**, the graph is called a **k-regular graph**.

Example



If each vertex connects to **2 edges**, the graph is **2-regular**.

Example graph:

A—B
| |
D—C

Each vertex has **degree = 2**.

Key Points

- All vert ces have equal number of edges.
- Degree of every vertex is constant.

Examples of Regular Graphs

Graph	Degree
Cycle graph C_4	2-regular
Cube graph	3-regular

---

2. Connected Graph:

Def nit on : A graph is **connected** if there exists at least one path between every pair of vert ces.

Meaning:
Every vertex can reach every other vertex.

Example

A—B—C
|
D

All vert ces are reachable from one another.

Key Idea : If you can travel from **any node to any other node**, the graph is connected.

Opposite Case

If some vert ces cannot reach others → **Disconnected graph**.

Example:

A — B C — D

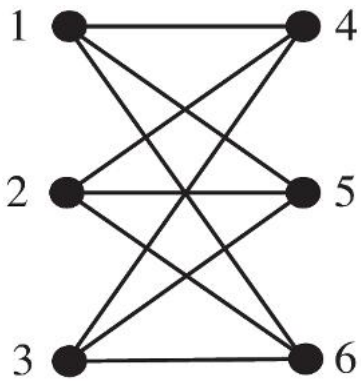
Two separate components.

---

3. Bipart te Graph

Def nit on (Exam Ready)

A **bipart te graph** is a graph whose vertex set can be divided into two disjoint sets such that every edge connects a vertex from one set to a vertex in the other set.



Structure

Vert ces split into two groups:

Set **U**
Set **V**

Edges only go **between sets**, not inside the same set.

Example

U = {A, B}
V = {C, D}

Edges:

A — C
A — D
B — C

Key Rule : No edge exists between vert ces within the same set.

Real Life Example

- Job assignment (workers ↔ jobs)
- Students ↔ courses

---

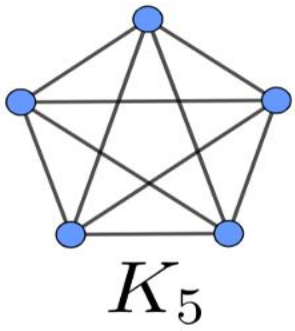
4. Complete Graph

Def nit on: A **complete graph** is a simple graph in which every pair of dist nct vert ces is connected by an edge.

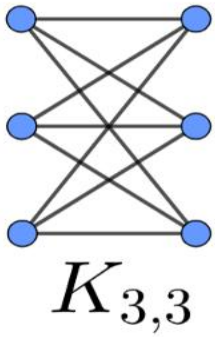
Notat on: Complete graph with **n vert ces**:

$$K_n$$

Example



K_5



$K_{3,3}$

For **4 vert ces**:

K_4

Each vertex connects to all others.

Number of Edges

Formula:

$$E = \frac{n(n-1)}{2}$$

Example: K_4

$$E = \frac{4(3)}{2} = 6$$

Key Idea : Maximum number of edges possible in a simple graph.

Quick Memory Trick

Regular → same degree
Connected → reachable nodes
Bipart te → two groups
Complete → all connected

Theorem 1 — Handshaking Theorem (Graph Theory)

Statement : For any graph G with vert ces v1,v2,…,vn and edge set E, the sum of the degrees of all vert ces is equal to **twice the number of edges** in the graph.

Mathemat cally:

$$\deg(v_1) + \deg(v_2) + \cdots + \deg(v_n) = 2|E|$$

Or

$$\sum_{i=1}^{n} \deg(v_i) = 2|E|$$

Where,

- $\deg(v_i)$ = degree of vertex v_i
- |E| = number of edges in graph

Intuit on Behind the Theorem

Each **edge connects two vert ces**.

So every edge contributes **1 degree to each endpoint**.

Therefore:

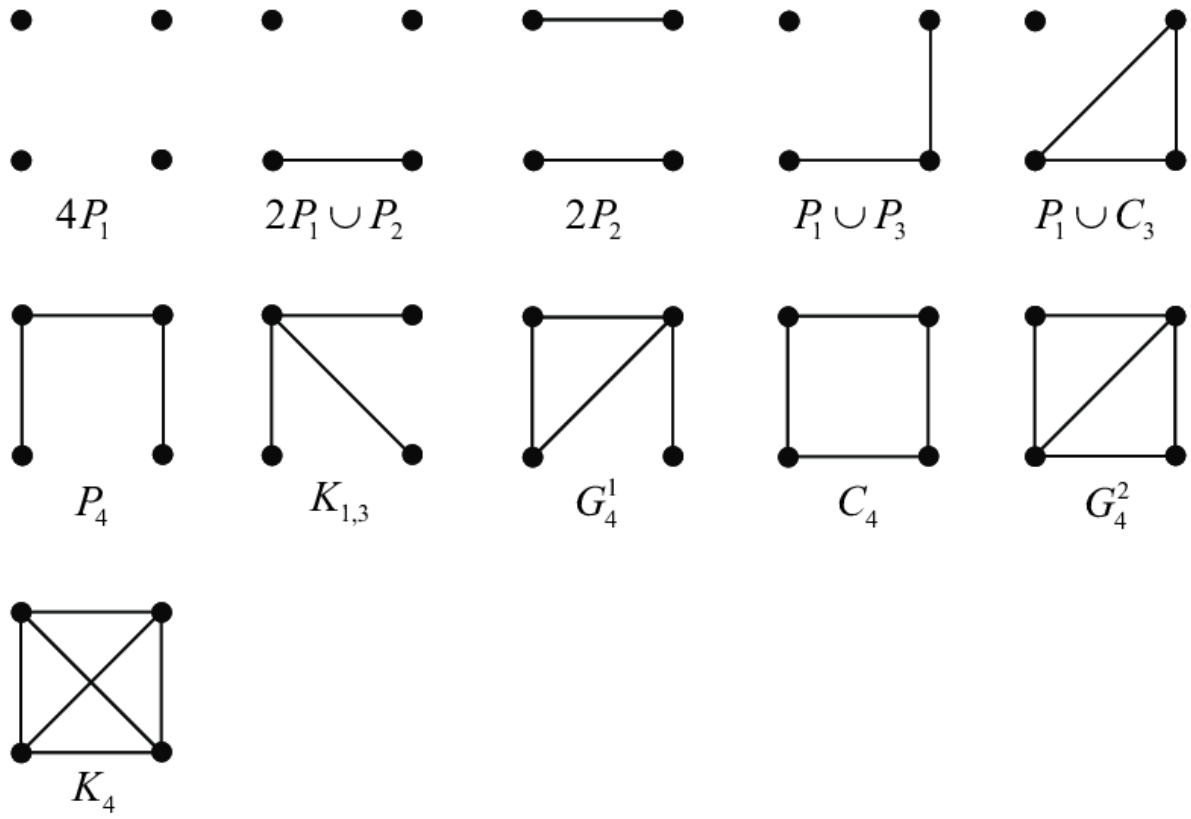
- One edge contributes **2 degrees** total.

Hence:

Total degree count = **2 × number of edges**.

Example

Consider graph:



Suppose graph:

Vert ces:
A, B, C, D

Edges:
AB, BC, CD, DA

Number of edges:

|E|=4

Step 1 — Calculate Degrees

Vertex	Degree
A	2
B	2
C	2
D	2

Step 2 — Sum of Degrees

$$2+2+2+2=8$$

Step 3 — Apply Theorem

$$2|E| = 2 \times 4 = 8$$

Both sides equal.

Theorem verif ed.

Important Consequence of the Theorem

From this theorem we get another result:

The number of vert ces with **odd degree** in a graph is always **even**.

Example: Odd-degree vert ces must appear in pairs.

Why It Is Called Handshaking Lemma

Imagine a group of people shaking hands.

- Every handshake involves **two people**.
- If we count all handshakes per person, each handshake gets counted twice.

So total handshakes = half of total degree count.

Theorem 2 — Number of Odd Degree Vert ces

Def nit on: A vertex of a graph is called an **odd vertex** if the degree of that vertex is an odd number.

Example: If degree of vertex = **1, 3, 5, 7**, etc., then it is an **odd vertex**.

Theorem Statement : In any graph G, the number of vert ces having **odd degree is always even**.

Proof

From the **Handshaking Theorem**:

$$\sum_{i=1}^{n} \deg(v_i) = 2|E|$$

Since 2|E| is **even**, the sum of the degrees of all vert ces in a graph is **even**.

Now divide vert ces into two groups:

1. Vert ces with **even degree**
2. Vert ces with **odd degree**

- Sum of degrees of even vert ces → **even number**
- Sum of degrees of odd vert ces → depends on count of odd vert ces

Property of numbers:

- Sum of an **odd number of odd integers** = odd
- Sum of an **even number of odd integers** = even

Since total degree sum must be **even**, the number of odd-degree vert ces must be **even**.

Therefore,

The number of vert ces with odd degree in any graph is always even.

Conclusion : Number of odd-degree vert ces is always even

---

1 Euler Graph

Def nit on : A graph is called a **Euler graph** if it contains a closed path that uses every edge of the graph exactly once.

This path is called an **Euler circuit**.

Euler Circuit : A path that:

- starts and ends at the same vertex
- uses every edge **exactly once**

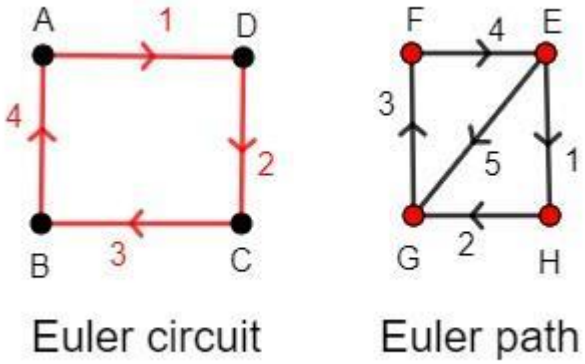
Euler Path : An Euler path is a path that uses every edge of the graph exactly once but does **not necessarily start and end at the same vertex**.

Euler Graph Condit ons

A connected graph is Eulerian **if and only if**:

1. Every vertex has **even degree**
2. The graph is **connected**

Example



Euler circuit Euler path

Example graph:

A — B
| |
D — C

Degrees:

Vertex	Degree
A	2
B	2
C	2
D	2

All degrees even → **Euler graph**

Possible Euler circuit:

A → B → C → D → A

Hamiltonian Graph

Def nit on : A graph is called a **Hamiltonian graph** if it contains a cycle that visits every vertex exactly once.

This cycle is called a **Hamiltonian cycle**.

Important:

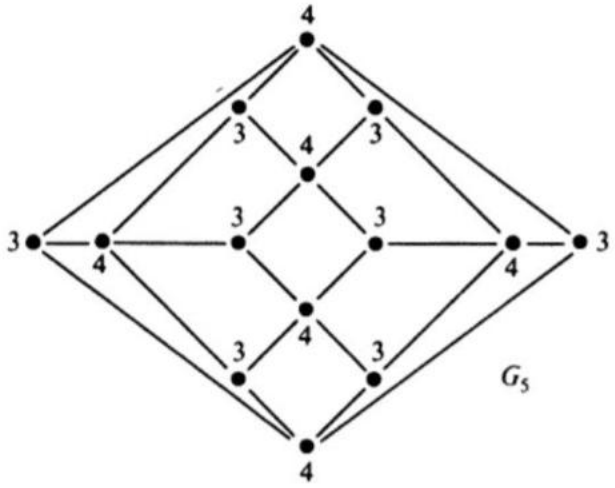
- Every **vertex visited once**
- Start and end vert ces are same

Hamiltonian Path

A Hamiltonian path is a path that visits every vertex exactly once but does not necessarily return to the start ng vertex.

Example

b) Show that there is no Hamiltonian cycle for the following graph G_5.



Example graph:

A — B
| |
D — C

Hamiltonian cycle:

A → B → C → D → A

Every vertex visited once.

Key Dif erence

Feature	**Euler Graph**	**Hamiltonian Graph**
Focus	Edges	Vert ces
Rule	Every edge used once	Every vertex visited once
Condit on	All vert ces even degree	No simple universal condit on

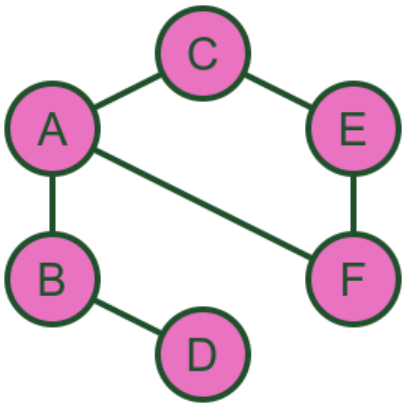
---

Simple Memory Trick

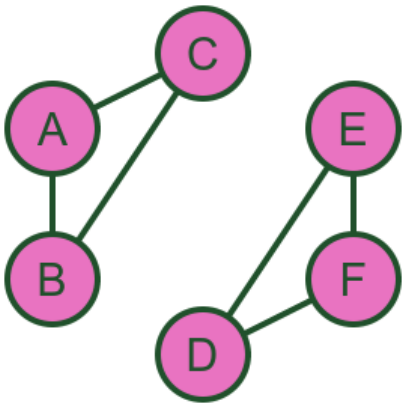
Euler → **Edges**

Hamilton → **Vert ces**

1 Connected Graph



Connected



Not connected

Def nit on: A graph is called **connected** if there exists at least one path between every pair of vert ces.

Meaning

Every vertex can reach every other vertex.

Example

Graph:

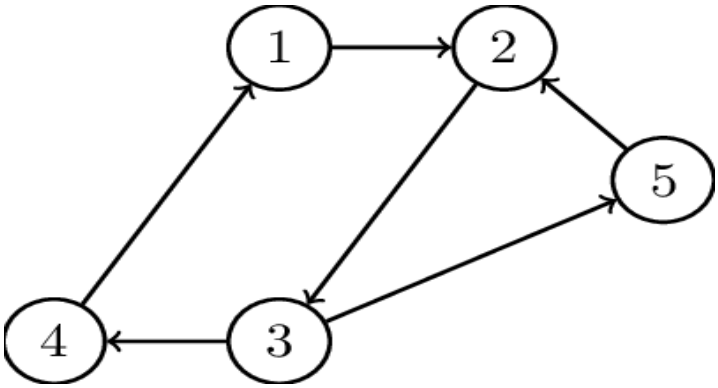
A — B — C
|
D

Paths exist between all vert ces.

So the graph is **connected**.

---

2 Strongly Connected Graph (Directed Graph)



Def nit on : A directed graph is **strongly connected** if there exists a directed path from every vertex to every other vertex.

Meaning

For vert ces **u and v**:

- u can reach v
- v can reach u

Example

A → B
B → C
C → A

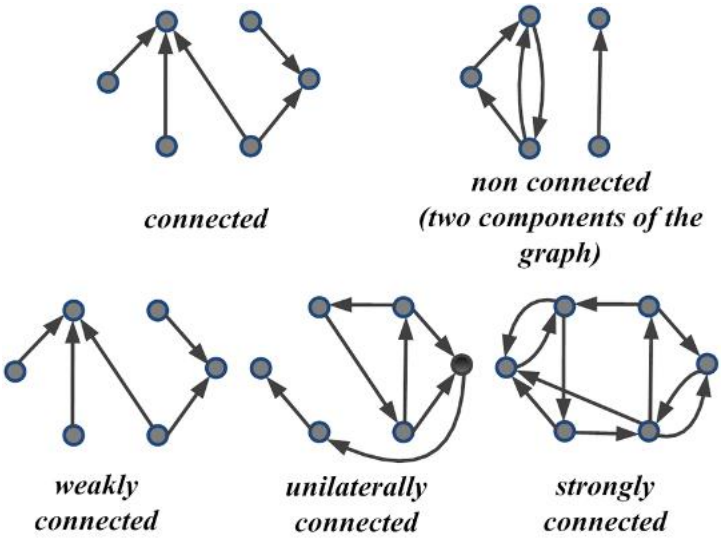
From any vertex you can reach all others.

Therefore **strongly connected**.

---

3 Weakly Connected Graph

Def nit on : A directed graph is **weakly connected** if replacing all directed edges with undirected edges results in a connected graph.



Meaning

Ignoring edge direct on → graph becomes connected.

Example

A → B
C → B

Ignoring direct on:

A — B — C

Graph becomes connected → **weakly connected**.

---

4 Unilaterally Connected Graph

Def nit on : A directed graph is **unilaterally connected** if for every pair of vert ces u and v, there exists a directed path from **u to v or from v to u**.

Meaning:

At least **one direct onal path exists**.

Not necessarily both.

Example

A → B → C

- A can reach C
- C cannot reach A

St ll **unilaterally connected**.

---

5 Components of a Graph

A **component** is a maximal connected subgraph.

Strong Component

A maximal subgraph in which every vertex is reachable from every other vertex.

Example:

A → B → C → A

These three form a **strong component**.

Weak Component

A maximal subgraph that becomes connected if edge direct ons are ignored.

Example:

A → B
C → B

Ignoring direct ons → connected.

Unilateral Component

A maximal subgraph in which every pair of vert ces is unilaterally connected.

Example:

A → B → C

Paths exist in one direct on.

Comparison Table

Type	Condit on
Connected Graph	Path between every pair of vert ces
Strongly Connected	Directed path exists both ways
Weakly Connected	Ignoring direct ons makes graph connected
Unilaterally Connected	Path exists in at least one direct on

Quick Example Comparison

Graph:

A → B → C

Type	Result
Strongly Connected	✗ No
Weakly Connected	✓ Yes
Unilaterally Connected	✓ Yes

Exam Tip

Students usually confuse **strong vs weak connect vity**.

Remember:

Strong → both direct ons
Weak → ignore direct ons
Unilateral → one direct on enough

Applicat on of Graphs in Operat ng Systems

Resource Allocat on and Deadlock Detect on

Graph theory is used in **operat ng systems** to represent how resources are allocated to processes and to detect **deadlocks**.

---

1 Resource Allocat on Graph (RAG)

Def nit on : A resource allocat on graph is a directed graph used to represent the allocat on of resources to processes in an operat ng system.

It helps determine whether the system is in a **deadlock state**.

Components of Resource Allocat on Graph

Component	Representat on	Meaning
Process	Circle	Program request ng resources
Resource	Square	System resource (CPU, printer, memory)
Request Edge	P → R	Process request ng resource
Assignment Edge	R → P	Resource assigned to process

Example

Example graph:

Processes
P1, P2

Resources
R1, R2

Edges:

P1 → R1 (request)
R1 → P2 (allocated)
P2 → R2 (request)

---

2 Deadlock

Def nit on : A deadlock is a situat on in which two or more processes are wait ng indef nitely for resources held by each other.

Meaning:
No process can proceed.

Example

P1 holds R1 and waits for R2
P2 holds R2 and waits for R1

Both wait forever → **deadlock occurs**.

---

3 Deadlock Detect on Using Graph

In a **resource allocat on graph**, deadlock appears as a **cycle**.

Example cycle:

P1 → R1 → P2 → R2 → P1

Cycle indicates deadlock.

---

4 Deadlock Detect on Rule

Condit on	**Result**
Graph has no cycle	No deadlock
Graph has cycle	Deadlock possible
Cycle with single resource instance	Deadlock occurs

---

5 Deadlock Prevent on :

Operat ng systems use strategies to avoid or break deadlocks.

Methods

1. Resource ordering
2. Resource preempt on
3. Deadlock detect on algorithms
4. Terminat ng processes

---

6 Example of Deadlock Correct on

Suppose:

P1 → R1 → P2 → R2 → P1

Break the cycle by:

- releasing a resource
- stopping one process

Then the graph becomes acyclic.

Deadlock resolved.

Key Exam Points

Always remember:

- **RAG represents resource allocat on**
- **Cycle detect on ident f es deadlock**
- Graph theory helps analyse **operat ng system behaviour**

Matrix Representat on of Graphs

Graphs can also be represented using **matrices**, which is useful in algorithms and computer implementat ons.

1 Matrix Representat on of a Graph

Def nit on : A matrix representat on of a graph is a way of represent ng a graph using a matrix to show the connect ons between vert ces.

If a graph has **n vert ces**, we form an **n × n matrix**.

Each row and column represent a vertex.

---

2 Adjacency Matrix

Def nit on : The adjacency matrix of a graph is a square matrix used to represent which vert ces are adjacent to which other vert ces.

Notat on:

$$A = [a_{ij}]$$

Where:

$$a_{ij} = \{1\ if\ there\ is\ an\ edge\ between\ v_i\ and\ v_j\ ,otherwise\ 0$$

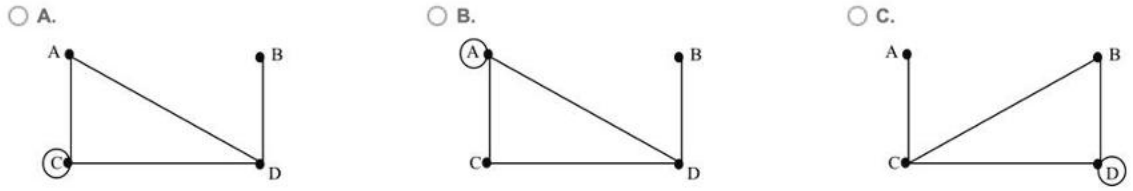
Example Graph

1. draw graph from descript ons:

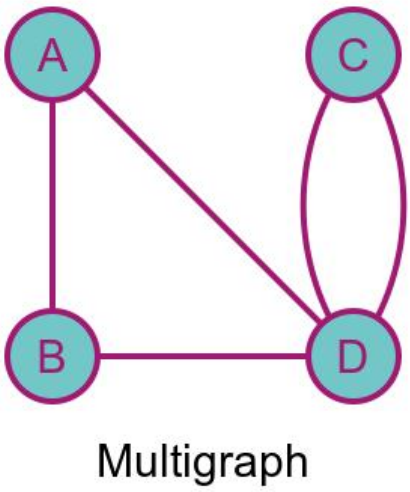
Draw a graph for the description.

The vertices are A, B, C, and D. The edges are AC, AD, BD, CD, and CC.

Choose the correct graph below.



2. make graph from adjency metrix :



	A	B	C	D
A	0	1	0	1
B	1	0	0	1
C	0	0	0	2
D	1	1	2	0

Vert ces

A, B, C, D

Edges

(A,B)
(A,C)
(B,D)

Adjacency Matrix

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	0
D	0	1	0	0

Explanat on

1 = edge exists
0 = no edge

---

3 Boolean (Bit) Matrix

Def nit on : A Boolean matrix is a matrix where entries are only **0 or 1** indicat ng absence or presence of edges.

Adjacency matrix itself is of en a **Boolean matrix**.

Values used:

Value	Meaning
1	Edge exists
0	No edge

So adjacency matrices are usually **Boolean matrices**.

---

4 Number of Paths Using Adjacency Matrix

If

$$A$$

is the adjacency matrix of graph G,

then

$$A^n$$

gives the number of **paths of length n between vert ces**.

Meaning:

$$(A^n)_{ij}$$

= number of paths from vertex **I** to vertex **j** of length **n**.

Example

Graph

A — B — C

Vert ces:

A, B, C

Step 1 — Adjacency Matrix

	A	B	C
A	0	1	0
B	1	0	1
C	0	1	0

Step 2 — Square the Matrix

$$A^2 = A \times A$$

Result

	A	B	C
A	1	0	1
B	0	2	0
C	1	0	1

Interpretat on

Example:

Entry (A,C) = 1

There is **1 path of length 2** from A to C.

Path:

A → B → C

Key Points

Concept	Meaning
Adjacency Matrix	Shows connect ons between vert ces
Boolean Matrix	Matrix with only 0 and 1
Matrix Power A^n	Number of paths of length n

---

1 Incidence Matrix

Def nit on : The incidence matrix of a graph is a matrix that shows the relat onship between **vert ces and edges**.

If a graph has:

- **n vert ces**
- **m edges**

Then the incidence matrix will be:

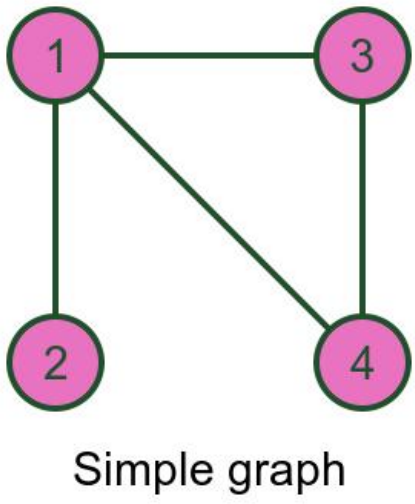
$$n \times m$$

Rows → vert ces
Columns → edges

Rule

Value	Meaning
1	Vertex is incident with edge
0	Vertex not incident with edge

Example Graph



Simple graph

Vert ces: A, B, C

Edges:

e_1 = (A,B)
e_2 = (B,C)
e_3 = (C,A)

Incidence Matrix

	e_1	e_2	e_3

A	1	0	1
B	1	1	0
C	0	1	1

Example:

Edge e_1 **(A,B)**

A = 1
B = 1
C = 0

---

2 Path Matrix

Def nit on: The path matrix of a graph shows whether a path exists between two vert ces.

Rule

Value	Meaning
1	Path exists
0	No path

Example Graph

A — B — C

Path Matrix

	A	**B**	**C**
A	1	1	1
B	1	1	1
C	1	1	1

Because every vertex is reachable.

---

3 Reachability Matrix (Path Matrix)

Def nit on : The reachability matrix of a graph shows whether a vertex can reach another vertex through any path.

It indicates **connect vity between vert ces**.

Construct on

Reachability matrix is obtained using:

$$R = A + A^2 + A^3 + \cdots + A^n$$

Where

A = adjacency matrix

Example

Graph

A → B → C

Adjacency Matrix

	A	B	C
A	0	1	0
B	0	0	1
C	0	0	0

Reachability Matrix

	A	B	C
A	0	1	1
B	0	0	1
C	0	0	0

Explanat on :

A can reach B and C
B can reach C

Warshall's Algorithm (Graph Theory)

Warshall's Algorithm is used to determine **reachability between vert ces** in a directed graph.

It computes the **transit ve closure** or **reachability matrix** of a graph.

---

1 Def nit on : Warshall's Algorithm is a method used to compute the reachability matrix of a directed graph from its adjacency matrix.

It determines **whether there exists a path between every pair of vert ces**.

---

2 Purpose of the Algorithm

Warshall's Algorithm helps to:

- Find **all possible paths between vert ces**
- Determine **reachability**
- Detect **connect vity in graphs**

---

3 Why It Is Important

Warshall's Algorithm is used in:

- Graph connect vity analysis
- Network rout ng
- Database dependency analysis
- Operat ng system resource graphs

---

4 Exam Pat ern

Students are usually asked:

1. Given adjacency matrix
2. Apply Warshall's algorithm step-by-step
3. Find **reachability matrix**

Summary

Step	Act on
1	Start with adjacency matrix
2	Use OR and AND rule
3	Check intermediate vert ces
4	Final matrix gives reachability